



# 11. Mídias

---

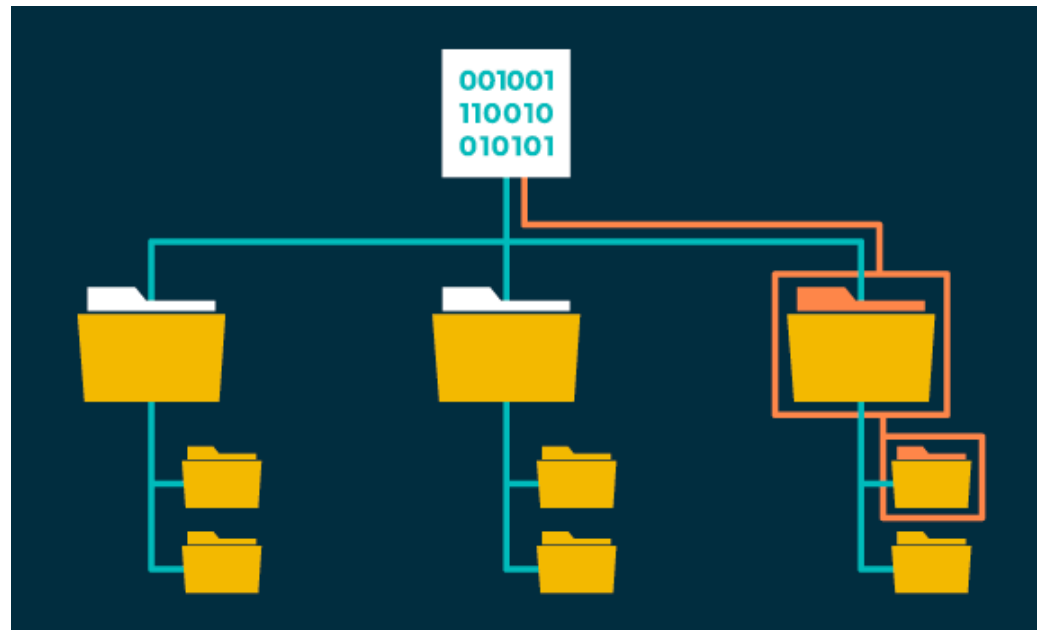
SCC0219 – Introdução ao Desenvolvimento Web

Prof<sup>a</sup>. Dr<sup>a</sup>. Bruna C. Rodrigues da Cunha

[brunaru@icmc.usp.br](mailto:brunaru@icmc.usp.br)

# Armazenamento de Arquivos

- O armazenamento de arquivos organiza os dados na hierarquia de documentos e arquivos em formatos de pasta.



# Armazenamento de Arquivos

- Não é uma boa ideia armazenar arquivos binários grandes no banco de dados devido ao overhead, velocidade de acesso, número de acessos, tamanho do arquivo
- Blob (*Objeto Binário Grande*), é uma massa de dados em formato binário que não está necessariamente em conformidade com nenhum formato de arquivo:
  - O arquivo será convertido para uma *string* no formato Base64
  - Adiciona um tempo de codificação e decodificação
- é melhor guardar a referência do local do arquivo!

# Armazenamento de Objetos

- Um objeto é um **arquivo** e seus **metadados** (dados que descrevem o arquivo)
  - O armazenamento de objetos armazena dados (como objetos) dentro de “baldes” (***buckets***)
  - Um *bucket* é um recipiente para objetos
  - O armazenamento de objetos oferece alta escalabilidade e fácil acesso aos dados
  - Solução ideal para armazenar **grandes volumes** de dados **estáticos**, especialmente dados não estruturados (**mídias**)
-

# Formas de Armazenamento

- O **armazenamento de arquivos** tradicional geralmente usa o sistema de arquivos de rede ou outros protocolos de rede comuns que são otimizados para baixa latência
- O **armazenamento de objetos** usa HTTP para acessar dados. Isso simplifica a recuperação de dados por meio de vários aplicativos diferentes e até mesmo navegadores da web
- As principais diferenças entre armazenamento de objetos e de arquivos são a estrutura de dados e a escalabilidade. O armazenamento de arquivos é organizado hierarquicamente em diretórios e pastas e segue rígidos protocolos de arquivos. O armazenamento de objetos usa uma estrutura simples com metadados e um identificador exclusivo para cada objeto.

# Formas de Armazenamento

| Armazenamento de arquivo                                                                                                                   | Armazenamento de objetos                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Os dados podem ser transferidos localmente.                                                                                                | Os dados podem ser compartilhados em várias áreas.                                                             |
| Funciona bem em arquivos menores.                                                                                                          | Funciona bem em dados de dimensão superior.                                                                    |
| O armazenamento é baseado na capacidade e é denominado como dispositivos de armazenamento secundário. Armazena cerca de 100~500 terabytes. | Oferece armazenamento secundário na nuvem e é extremamente confiável. Ele armazena um máximo de 500 petabytes. |
| Possui gerenciamento simplificado para transferência de arquivos e é difícil trabalhar fora do <i>data center</i> .                        | Oferece alto desempenho mesmo sendo feito para operar além do <i>data center</i> .                             |

# Upload no Disco (Armazenamento de Arquivo)

- Vamos utilizar a biblioteca **multer**.
- Vamos utilizando o axios no cliente para conversar com o nosso serviço.
- Multer é um *middleware* para Node.js feito lidar com multipart/form-data, que é usado principalmente para fazer upload de arquivos.

**npm i multer**

---

# Upload no Disco

/app.js (exemplo simples em um arquivo só)

```
import express from 'express'
import multer from 'multer'

const app = express()
app.use(express.static('public'))
app.use(express.static('uploads'))

app.listen(3000, function () {
  console.log('Servidor ouvindo na porta 3000.')
})
```

---



# Upload no Disco

/app.js (continuação...)

```
var storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, 'uploads/')
  },
  filename: function (req, file, cb) {
    cb(null, Date.now() + '-' + file.originalname)
  })
var upload = multer({ storage: storage })

app.post('/upload', upload.single('arquivo'), function (request, response, next) {
  console.log('Upload realizado.')
  console.log(request.file)
  response.send('http://localhost:3000/' + request.file.filename)
})
```

---

# Upload de Objeto

- Atualmente, o serviço de armazenamento de objetos mais famoso é o **Amazon S3**.
- O Amazon S3 tem um custo bastante acessível (porém tem um custo). Por isso vamos utilizar uma alternativa *Open Source*, o MinIO.
- O MinIO oferece armazenamento de objetos compatível com S3 de alto desempenho.



- Vamos utilizar a biblioteca multer, o multer-s3.
-

# Configuração do MinIO

- Instalar e executar o MinIO  
Server: <https://min.io/download?license=agpl&platform=windows>
- Exemplo no Windows: `C:\minio.exe server C:\Data --console-address ":9001"`
- O web console fica acessível na porta **9001**

```
MinIO Object Storage Server
Copyright: 2015-2024 MinIO, Inc.
License: GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
Version: RELEASE.2024-01-29T03-56-32Z (go1.21.6 darwin/arm64)

Status:          1 Online, 0 Offline.
S3-API: http://192.168.15.5:9000  http://127.0.0.1:9000

RootUser: minioadmin
RootPass: minioadmin
```

# | Configuração do Minio (S3)

1. Criar **bucket**
2. Gerar chave de acesso (**accessKeyld** e **secretAccessKey**)
3. Determinar política de acesso (*policy*)

# Minio

1. Criar um *bucket* (e.g., chamei o meu de *devweb*)


**MINIO**

**OBJECT STORE**


**LICENSE**

User

Object Browser

Access Keys

Documentation

Administrator

Buckets

Policies

Buckets

?

🔄

🔍 Search Buckets

🗪

🔄

🗑️

🔄

Create Bucket +

devweb

Created: Wed May 22 2024 16:56:31 GMT-0300 (Horário Padrão de Brasília)

Access: R/W

🗑️

Usage

9.1MiB

🗑️ Objects

5

# Minio

2. Gerar uma chave de acesso (lembre-se de guardar os valores da chave e do segredo!)


**MINIO**

**OBJECT STORE**


**LICENSE**

User

Object Browser

**Access Keys**

Documentation

Administrator

Buckets

Policies

Access Keys

Delete Selected

Change Password

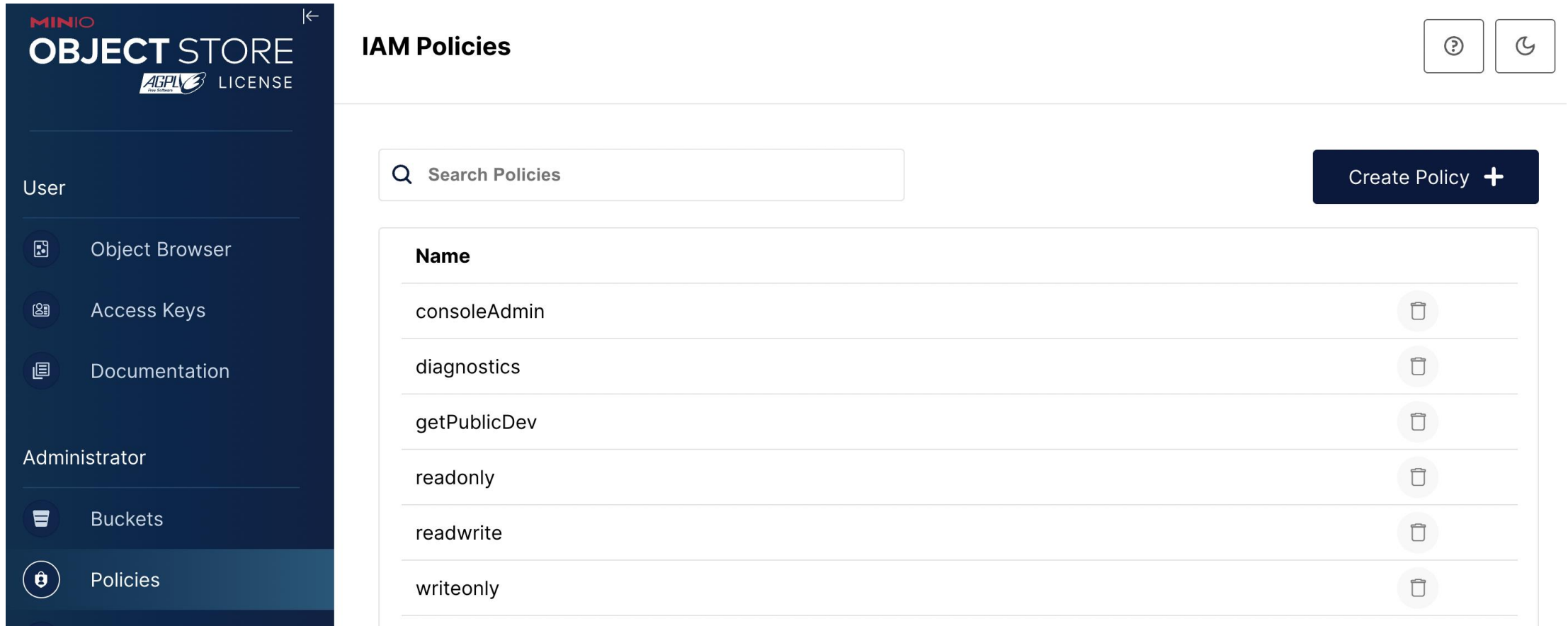
Create access key

| Select                   | Access Key       | Expiry | Status  | Name | Description                          |
|--------------------------|------------------|--------|---------|------|--------------------------------------|
| <input type="checkbox"/> | ywDsve79rRisy... |        | Enabled |      | <div> <div></div> <div></div> </div> |

# Minio

## 3. Determinar política de acesso (*policy*)

3.1. Criar uma política (e.g., *chamei a minha de **getPublicDev***)



The screenshot shows the Minio Object Store interface. On the left is a dark blue sidebar with the Minio logo and 'OBJECT STORE' text. Below the logo is the 'AGPL LICENSE' logo. The sidebar has two sections: 'User' with links for 'Object Browser', 'Access Keys', and 'Documentation'; and 'Administrator' with links for 'Buckets' and 'Policies' (which is highlighted). The main content area is titled 'IAM Policies' and features a search bar labeled 'Search Policies' and a 'Create Policy +' button. Below these is a table of existing policies.

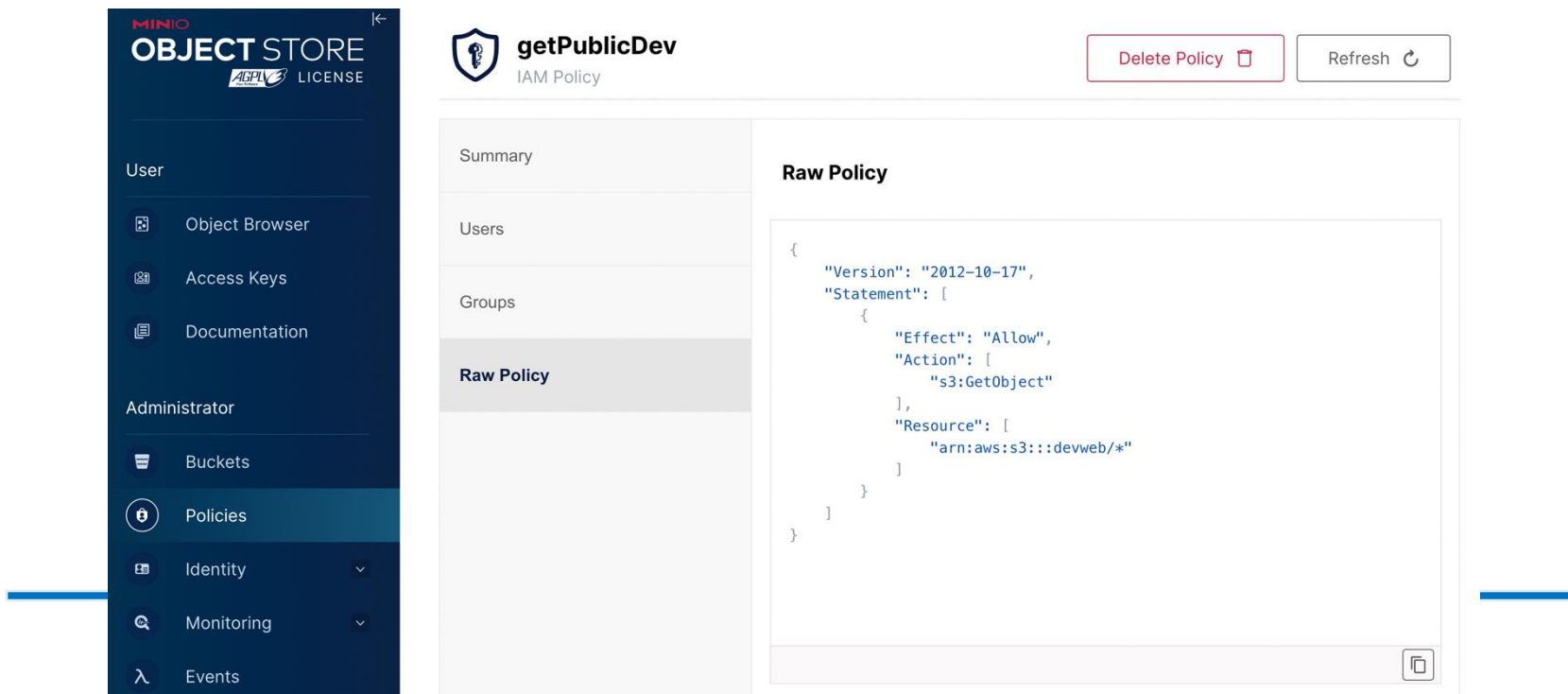
| Name         |                                                                                       |
|--------------|---------------------------------------------------------------------------------------|
| consoleAdmin |   |
| diagnostics  |  |
| getPublicDev |  |
| readonly     |  |
| readwrite    |  |
| writeonly    |  |

# Minio

## 3. Determinar política de acesso (*policy*)

3.1. Criar uma política (e.g., *chamei a minha de **getPublicDev***)

3.2. Determinar a política (nesse exemplo, recuperar as mídias é público, enquanto as outras operações são privado e necessita de chave, por padrão)



The screenshot displays the Minio Object Store console interface. On the left, a dark blue sidebar contains navigation links: User, Object Browser, Access Keys, Documentation, Administrator, Buckets, Policies (selected), Identity, Monitoring, and Events. The main content area is titled 'getPublicDev IAM Policy' and includes a 'Delete Policy' button and a 'Refresh' button. Below the title, there are tabs for 'Summary', 'Users', 'Groups', and 'Raw Policy' (selected). The 'Raw Policy' tab shows the following JSON configuration:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::devweb/*"
      ]
    }
  ]
}
```



# Minio

## 3. Determinar política de acesso (*policy*)

3.1. Criar uma política (e.g., *chamei a minha de `getPublicDev`*)

3.2. Determinar a política (nesse exemplo, recuperar as mídias é público, enquanto as outras operações são privado e necessita de chave, por padrão)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": [
        "arn:aws:s3:::devweb/*"
      ]
    }
  ]
}
```

---

# Upload de Objeto

- Vamos utilizar a biblioteca multer, o multer-s3 (além do express, previamente instalado).
- Precisamos também das bibliotecas do S3 feitas pela Amazon (aws-s3).

**npm i multer**

**npm i multer-s3**

**npm i '@aws-sdk/client-s3'**

---

# Upload de Objeto

/app.js (exemplo simples em um arquivo só)

```
import { S3Client, GetObjectCommand } from '@aws-sdk/client-s3'
import { getSignedUrl } from '@aws-sdk/s3-request-presigner'
import express from 'express'
import multer from 'multer'
import multerS3 from 'multer-s3'

const app = express()

app.use(express.static('public'))

app.listen(3003, function () { console.log('Servidor ouvindo na porta 3003.') })
```

---

# Upload de Objeto

/app.js (exemplo simples em um arquivo só)

```
const s3 = new S3Client({
  endpoint: 'http://127.0.0.1:9000',
  credentials: {
    accessKeyId: 'ywDsve79rRisyEYQYk4J',
    secretAccessKey: 'v4AYyC3fYKwZSW5PpypXwqHLApumqKuUUusyPL5AL', },
  sslEnabled: false, s3ForcePathStyle: true, region: 'us-east-1', })

const uploadArquivo = multer({ storage: multerS3({ s3: s3, bucket: 'devweb',
acl: 'public-read',
  key: function (request, arquivo, cb) {
    cb(null, Date.now() + '-' + arquivo.originalname)
  }})
})
```

---

# Upload de Objeto

/app.js (exemplo simples em um arquivo só)

```
app.post('/upload', uploadArquivo.single('arquivo'), function
(request, response, next) {
  console.log('Upload realizado.')
  const imgUrl = 'http://127.0.0.1:9000/' + 'devweb/' +
request.file.key
  response.send(imgUrl)
})
```

---

# Upload de Objeto

/public/index.html

```
<!DOCTYPE html>
<html lang="pt-br">
<head> <link rel="stylesheet" href="./css/style.css"> </head>
<body>
  <h1>Upload</h1>
  <form id="form-arquivo">
    <label for="file">Enviar um arquivo: </label>
    <input type="text" name="desc" placeholder="Descrição..." />
    <input type="file" name="arquivo">
    <button type="submit" class="button" onclick="enviar(event)">Enviar</button>
  </form>
  <div id="resposta"></div>
</body>
<script src="https://unpkg.com/axios/dist/axios.min.js"></script>
<script src="./js/script.js"></script>
</html>
```

---

# Upload de Objeto

/public/js/script.js

```
function enviar(e) {  
  e.preventDefault()  
  const form = document.getElementById('form-arquivo')  
  const formData = new FormData(form)  
  axios.post('http://localhost:3003/upload', formData, {  
    headers: { "Content-Type": "multipart/form-data", },  
  }).then((res) => {  
    const endereco = res.data  
    const img = document.createElement('img')  
    img.setAttribute('src', endereco)  
    img.setAttribute('width', 450)  
    document.getElementById('resposta').appendChild(img)  
  }).catch((err) => { console.log(err) })  
}
```

---